Byeharu — Project Report

byeharu — generated 2026-07-01 from BYEHARU_PROJECT_GUIDE.md · main 946cdc9 · head 0072

Byeharu — Project Guide (Plain-English Companion)

> **What this document is.** A plain-English tour of how Byeharu was built, why it was built in that order, what each piece means *in gameplay terms*, and where the project is heading next. It is written for a human who wants to *understand the project*, not to implement it.
>
> **What this document is NOT.** It is **not** an implementation specification and it does **not** replace the authoritative documents. When this guide and the specs disagree, the specs win. The sources of truth remain:
> - `docs/ROADMAP.md` — the forward direction and the numbered Phase plan.
> - `docs/MAINSHIP_TRANSITION.md` — the main-ship transition, OSN, and Repair & Recovery.
> - `docs/ACTIVITIES.md` — the expedition-activity abstraction.
> - `docs/ARCHITECTURE.md` / `docs/SYSTEM_BOUNDARIES.md` — engine design and ownership law.
> - `docs/DEV_LOG.md` — the dated, blow-by-blow history.
>
> **Status labels used throughout.** Every notable claim is tagged:
> - **[Implemented]** — built, deployed, and verified.
> - **[Designed]** — designed and/or approved, but **not** implemented.
> - **[Future]** — intended direction; details may still be undecided.
>
> Where a design is genuinely undecided, this guide says so rather than pretending certainty.

---

1. Title and purpose

Byeharu is a **server-authoritative PvE space-strategy game**. "Server-authoritative" means the server decides everything that matters (where your ship is, who wins a fight, what loot you earned); the browser only *shows* you the result and *asks* the server to do things. This makes the game cheat-resistant by construction — the client can never simply declare itself the winner.

This guide explains the project's **evolution** (how it grew from an empty shell to its current shape) and its **intended architecture direction** (where it is deliberately heading). It favors plain language first; technical terms are explained the first time they appear and collected in the **Glossary** (§12).

---

2. Current project snapshot

> **2026-06-30 update.** Production migration head is now `0072`; `main` head is `946cdc9`. **OSN port-to-port travel is ENABLED (`mainship_space_movement_enabled = true`).** Free arbitrary-coordinate travel is built but DARK**:** `mainship_coordinate_travel_enabled = false` **(server gate,** OSN-COORD-GATE-1 / `0070` **) so the raw coordinate command rejects and the server-derived** `coordinate_travel_available = false` for every caller. The frontend coordinate UI is now driven SOLELY by that runtime capability (**OSN-COORD-ENABLE-1B/1C**, migrations `0071`, PRs #57–#59) — the old `OSN_COORDINATE_TRAVEL_ENABLED` compile-time const is retired, so flipping the server flag later lights up the UI with **no redeploy**; until then it stays hidden. `mainship_send_enabled = true`. The three starter ports (Haven Reach, Slagworks Anchorage, Driftmarch Waypost) are **active/public**; **Phase 9** dock-service read surface (`get_my_current_dock_services()` + `DockServicesPanel`, `0069`) is **[Implemented]**. **PORT-ENTRY-1 (migration `0072`, PR #61) is DEPLOYED** — first-ship commissioning (`commission_first_main_ship()`, docked at Haven Reach), same-location normalization (`normalize_main_ship_dock()`), and the service-role-only `port_entry_commission_writer(uuid)`; additive function-only, concurrency-proven, **no player-facing UI yet**. A dedicated read-only production verifier (**PORT-ENTRY-1-VERIFY-1**, PR #62) is **merged but not yet run**. **Phase 10 Trading V1 is** fully designed/calibrated but NOT built**. Immediate next work: the player-facing** Claim First Ship **+** Finish Docking (normalize) **UI, then** Trading V1**. The detailed bullets below predate this — see `DEV_LOG.md` (**newest 2026-06-30 entry**) for the authoritative current state and forward plan.

> **Immediate next steps (approved direction, not started):** (1) main-ship provisioning + canonical port-entry transition (the Trading prerequisite); (2) Trading V1 implementation (read model → catalog seed → atomic buy/sell write path → Market UI → gated deploy) after the open product decisions are approved; then (3) Exploration → Mining → Modules/Captains → Ranking. `world_sites`, Online Presence, main-ship combat, and a cargo-loss/repair-cost redesign remain deferred.

As of this writing:

- **Branch / commit:** `main` equals `origin/main` (nothing unpushed), working tree clean. The last

code / deploy baseline is the OSN-3 **S6B4** merge `adc7009` (code commit `777fbd1`); the migration baseline is unchanged at **0060** (S6B is **frontend-only** — no migration); any commits on `main` above it are **documentation-only** closure records.

- **Database migrations:** applied through **0060** (`osn3_s6a_public_space_move_command`) — unchanged by S6B.
- **Two feature flags:** `mainship_send_enabled` is `true` on live (2026-06-21) — a controlled,

reversible activation of the **legacy named-location** main-ship send/move/return path only; and `mainship_space_movement_enabled` **remains** `false` (gates the coordinate-domain movement — the internal writer (S3), the arrival processor (S4), and now the **public, authenticated command wrapper** `command_main_ship_space_move` (S6A) all exist but stay **dark** behind this flag, which was **not** touched; with it false the public wrapper returns `feature_disabled` and writes nothing). The legacy send flip enables only the established named-location send UI; it does **not** enable any coordinate-movement or OSN player command. Rollback is the same controlled workflow (`dev-mainship-flag.yml`) with `mainship_send_enabled=false`.

- **OSN-1 / OSN-2 (a+b) are [Implemented], flag-gated.** The single map-marker resolver draws your

main ship and now understands the durable open-space position model (`spatial_state` + `space_x/space_y`). **OSN-3 S1** (schema + read-model), **OSN-3 S2** (the private, server-only transition boundary — lock/validate/resolve-origin/cross-domain-exclusion helpers), and **OSN-3 S3** (one private, `service_role`-only, flag-dark coordinate-movement *writer* `mainship_space_begin_move` that composes the S2 boundary) are also [Implemented]. **OSN-3 S4** (the cron-driven coordinate-arrival processor), **S5** (coordinate-complete destruction), and **S6A** (the **public, authenticated, flag-dark** coordinate-command wrapper `command_main_ship_space_move` that delegates to the private writer — the first player-facing boundary) are [Implemented] too — **still no map UI, no target selection, no player CTA, and the coordinate flag stays off**.

- **OSN-3 S6B (S6B1–S6B4) is [Implemented]** — a **read-only, frontend** fixed-space coordinate

foundation: a pure fixed-domain transform (`openSpaceTransform`: `worldToViewBox` / `worldToScreen` over the fixed `[-10000,10000]` world), provenance routing of the ship's open-space states through it (the discriminated `coordinateSpace`), a **development-only** preview that is **compile-time eliminated** from production bundles, and acceptance that the **real** `MainShipMarker` fixed route + the preview **co-move** under the camera. Still **no** map command UI, tap selection, target persistence, or flag flip. The fixed-space ↔ named-location **presentation** (**S6B-PRES**) is the **mandatory gate before any S6D enablement**.

- **One main ship per player** is a durable design fact: the `main_ship_instances` table holds

exactly one ship row per player today (enforced by a uniqueness rule on `player_id`). Multiple ships per player is a deliberately deferred future step.

Three categories you will see repeatedly — keep them distinct:

Category	Meaning	Examples in Byeharu
Implemented systems	Built, deployed, verified, live (some gated behind a flag)	The expedition engine (travel/combat/return), inventory, the galaxy map, the main-ship instance, the OSN-1 marker, OSN-2 (durable open-space position model), OSN-3 **S1** (coordinate-domain schema + read-model), OSN-3 **S2** (server-only transition boundary — lock/validate/resolve-origin/exclusion helpers), OSN-3 **S3** (one private, service_role-only, flag-dark coordinate-movement writer `mainship_space_begin_move`), OSN-3 **S4** (one private, service_role-only, cron-driven coordinate-arrival processor `process_mainship_space_arrivals`), OSN-3 **S5** (coordinate-complete trusted destruction primitive `dev_set_main_ship_destroyed`), OSN-3 **S6A** (the public, authenticated, flag-dark coordinate-command wrapper `command_main_ship_space_move` that delegates to the private writer), OSN-3 **S6B** (S6B1–S6B4: read-only frontend fixed-space coordinate foundation — pure transform, provenance routing of the ship's open-space states, a dev-only compile-time-eliminated preview, and the real fixed-route + camera-co-move acceptance)
Design-only work	Decided/approved on paper, **not** built	OSN-3 follow-ups: **S6B-PRES** (the **mandatory** pre-S6D fixed-space ↔ named-location presentation decision — named locations through a verified fixed-domain transform **or** a separate coordinate map mode where legacy markers are hidden/non-spatial), **S6C** (tap-to-target + confirm UI), **S6D** (controlled enablement); OSN-4 Stop; final Repair & Recovery
Future initiatives	Intended, but not yet fully designed	OSN-5, Exploration/Mining/Trading, Online Presence, player interaction, main-ship combat

3. Big-picture game vision

The game Byeharu wants to become, in one breath:

> **You own and develop a single main ship.** You send it out across a shared galaxy to **explore** unknown space, **mine** resources, **trade** goods between markets, and **fight pirates**. Your ship returns home with loot, profit, data, and discoveries; you spend those to grow stronger — better **captains**, better **modules**, **support craft**. Over time the game opens up to **other players** (seeing them, then trading/cooperating/fighting them) and to **rankings** that measure who plays best.

Core fantasy: "*my ship and crew go on dangerous expeditions, return with rewards, and become stronger.*"

The direction of travel. Byeharu started life as a more traditional **fleet-and-named-location** game: you pick a destination from a list of named places and send a stack of disposable units there. It is deliberately **transitioning toward a main-ship-centered, open-space strategy game**: one persistent ship that is the emotional center, moving through *continuous space* rather than hopping between menu entries. That transition is the throughline behind most of the recent work — and it is exactly why **Open-Space Navigation (OSN)** (§7) is now the central next direction.

---

4. Milestones M1 through M7

The first era of development was organized as **milestones** (M1–M7). These built the **engine** — the reusable travel → fight → return → reward loop that every future activity will sit on top of. All of M1–M7 are **[Implemented]** and verified.

> **A naming note before we start.** The milestone numbers are *roughly* chronological but not perfectly: **M7** (ship production) was built first, then a follow-up reframed and corrected it into the **M4.5 "Serial Build Queue Foundation."** So M4.5 is numbered as if it sits between M4 and M5, but in calendar time it was finalized *after* M7. This guide presents them in their conceptual order and flags the overlap. (See also §5's larger naming-conflict discussion.)

M1 — Project shell [Implemented]

- **Problem it solved:** there was no project. We needed a clean, modern web app with login.
- **Systems added:** the React + TypeScript + Vite front end, Tailwind styling, Zustand state,

and the Supabase backend (Postgres database, Auth, row-level security, server functions, scheduled jobs). A `profiles` table with per-user access rules and an auto-create-profile trigger.

- **Gameplay meaning:** you can sign up and log in. Nothing to *do* yet.
- **Technical principle:** server-authoritative from day one; the client never holds secrets.
- **Did not solve:** any actual game (no world, no ships, no movement).

M2 — Read-only world map [Implemented]

- **Problem:** the game needs a *place*. A galaxy to look at.
- **Systems added:** the static world tables — **sectors** → **zones** → **locations** (a 3-level

hierarchy) — seeded with a small starter galaxy, plus a single read function (`get_world_map`) and a basic map screen. Locations have a type (`safe_zone` , `pirate_hunt`) and coordinates.

- **Gameplay meaning:** you can open a map and see named places. You cannot interact with them yet.
- **Technical principle: strict system boundaries** were written down first (one "owner" per

table; systems talk only through approved functions). The Map can be *read* by anyone but *written* by no client.

- **Did not solve:** movement, danger, or anything dynamic. The map is a read-only backdrop.

M3 — Fleets, movement, presence, server processing [Implemented]

- **Problem:** the world is static. We need to *go* somewhere and have the server track it.
- **Systems added:** the **base** system (your home + your units + your resources), the **fleet**

system (a group of units you send out), the **movement** system (origin/target coordinates, depart/arrive times, travel speed), and the **presence** system (the fact that a fleet "is at" a location). A **scheduled job runs every 30 seconds** to resolve arrivals and returns. A "Command Center" front

end lets you actually send a fleet and watch the countdown.

- **Gameplay meaning:** send units to a safe location, watch them travel, arrive, leave, and come

home — the **movement spine** of the whole game.

- **Technical principle:** movement and (later) combat are **separate systems bridged by

presence**. Proving the harmless movement loop first means later combat bugs are isolated to combat.

- **Did not solve:** combat. M3 deliberately only travels to *safe* places.

M4 — Server-authoritative pirate combat [Implemented]

- **Problem:** travel is safe and boring. We need danger and rewards.
- **Systems added: pirate combat** that resolves on the server in fixed **3-second ticks**,

with rising waves, per-unit hull HP and losses, a **retreat** action (you still take damage while disengaging), defeat handling, and **metal rewards**. Critically, the **reward-on-arrival law**: loot is *pending* while you are out and is **only secured when your fleet reaches home** — lose before you get home and you forfeit it. A combat report records what happened.

- **Gameplay meaning:** the real loop appears — risk a fight, survive, bring loot home, profit.
- **Technical principle:** the server owns **every** combat outcome; the client only animates

cosmetic effects and may press "retreat." A security pass (`0021`) **locked down internal functions** so a clever client cannot call them directly to cheat.

- **Did not solve:** any non-combat activity; deep balance; main-ship combat (this is *unit-stack*

combat, the older model).

M4.5 — Ship production (Serial Build Queue Foundation) [Implemented]

- **Problem:** the only way to get units was the starter grant. We need to *build* more, and the

first version accidentally built everything in parallel.

- **Systems added:** a **serial build queue** — you spend metal to queue ship training; **one

order builds at a time**, the rest wait without ticking; you can cancel (full refund while waiting, partial while active). A scheduled job completes finished orders and starts the next.

- **Gameplay meaning:** a spending loop — turn metal into more units, on a timer.
- **Technical principle:** this is the **reusable production foundation**. Its real future meaning

is "build support craft / modules / repair kits / equipment" — the *same queue*, different outputs.

- **Did not solve:** buildings, shipyards, research, multi-resource costs (all deferred).
- **Naming note:** M4.5 is the **corrected, serial** version of what M7 first built (below).

M5 — Living-world danger / pressure [Implemented]

- **Problem:** the world doesn't react. Every location feels the same forever.
- **Systems added: world-state** tables (`location_state` , `zone_state`) tracking *pressure*

and a *danger modifier*, updated by a **60-second** scheduled job. Pressure **decays toward a baseline** so that, with nobody playing, locations drift back to normal instead of maxing out and punishing newcomers. Hunting relieves pressure.

- **Gameplay meaning:** locations have a living "pirate activity / danger" level that combat reads.
- **Technical principle: World State is the sole writer** of location/zone state; combat may

only *read* the danger modifier. At baseline the modifier is exactly 1.0, so M4's balance is unchanged until pressure actually drifts.

- **Did not solve:** player-driven economy, events, or newbie-safe zones (decay was the chosen fix).

M6 — Frontend depth [Implemented]

- **Problem:** the backend was rich but the player couldn't *see* it clearly.
- **Systems added (front end only):** a location detail panel, a readable **round-by-round combat

log**, a `/reports` page for past battles, pre-dispatch danger warnings, and clearer fleet lifecycle wording.

- **Gameplay meaning:** the player can finally understand danger, combat, and outcomes at a glance.
- **Technical principle: read-only** — M6 changed no backend logic, combat math, or rewards.
- **Did not solve:** anything new mechanically; it is a clarity pass.

M7 — Training-first ship production [Implemented]

- **Problem:** the economy was "combat rewards only." We needed a way to *spend* metal to grow.
- **Systems added:** the first **ship-training** system — spend metal → queue training → a

scheduled job completes ships into your base. (This is the system that M4.5 then corrected into a proper **serial** queue and reframed as the Serial Build Queue Foundation.)

- **Gameplay meaning:** the spending half of the economy loop.
- **Technical principle:** Production only writes its own order table; it **spends and deposits

through the Base system**, never reaching into other systems' data.

- **Did not solve:** parallel-vs-serial correctness (fixed in M4.5), buildings/research/trade.
- **Naming note: M7 and M4.5 describe the same production system at two stages** — M7 = first

build, M4.5 = the serial correction + foundation reframe.

---

5. The main-ship transition and Phases 1 through 10H

After the engine (M1–M7) was solid, the project entered a second era organized as **numbered Phases**. The goal: **reframe the proven engine around a single persistent main ship** without throwing the engine away. The guiding rule was "*don't replace the engine — replace the **source** of the expedition's stats.*" Combat, movement, and rewards stay; what changes is **who** goes on the expedition (one main ship + captains + modules, instead of a disposable unit stack).

> ### ⚠️ Read this before the phase list — historical naming is genuinely tangled
>
> Two **different** numbering schemes both use "Phase 10," and the main-ship transition reused some numbers for content different from what was originally planned. To avoid confusion:
>
> 1. `ROADMAP.md` **Phase numbers** (Phase 10 = *Trading*, 11 = *Exploration*, 12 = *Mining*, …) are the **long-term product plan**. These are mostly **[Future]**.
> 2. **Main-ship "Phase 10A–10H"** are the **transition work** described here. They **collide numerically** with the ROADMAP's "Phase 10," but they are a *different thing*.
> 3. **What was built under 10E/10F is NOT what the original design doc planned for 10E/10F.** The plan said 10E = main-ship *combat + destruction* and 10F = *deprecate the old send path*. In reality 10E became *legacy isolation hardening* and 10F became the *destroyed/repair safelock*. **Main-ship combat was never built.**
> 4. **There is no delivered "10G."** 10G is the *planned multi-ship* step, still unbuilt.
> 5. **"10I" / "10J" never existed as real phases** — they were conversation-only labels that were dropped. The open-space work is now called **OSN** (§7), deliberately *not* a Phase number, so it collides with neither scheme.
>
> The detailed reconciliation lives in `MAINSHIP_TRANSITION.md` §7. Historical labels are preserved, not retroactively renamed.

The phases

Phase	What it did	Status & current relevance
1 — Roadmap reconciliation	Wrote `ROADMAP.md`; reframed M2–M4 as the "Expedition Engine" and M4.5 as the "Build Queue Foundation." Docs only.	**[Implemented]** (docs). The direction-setter everything else follows.
2 — Activity design	Wrote `ACTIVITIES.md`: a clean way to add new activity types (hunt/trade/explore/mine) without a giant tangled switch. Docs only.	**[Implemented]** (docs). The blueprint for future Exploration/Mining/Trading.

3 — Generic inventory	Added `item_types`, `player_inventory`, `inventory_ledger` + deposit/spend/balance functions. Metal stays in base resources; *items* live in inventory.	**[Implemented]**. The storage layer all future loot flows into.
4 — Pending loot bundle	Generalized the reward into a `{ metal?, items[] }` bundle that rides home and is split on arrival (metal → base, items → inventory). No schema change.	**[Implemented]**. Preserves the reward-on-arrival law for items too.
5 — Multi-item pirate loot	Pirates now drop real items (deterministic loot table) alongside metal, secured on return, forfeited on defeat.	**[Implemented]**. First real item drops.
6 — Support-craft metadata	Added a catalog of "support craft" (escort, cargo drone, miner, etc.) as *capacity-limited loadout choices, not additive power*.	**[Implemented] but later DEPRECATED.** The vision moved to *captains + modules + upgrades*; support craft is now dormant scaffolding, hidden from the UI, not deleted.
7 — Main ship instance	Created `main_ship_instances` (one ship per player) + a starter hull. The ship exists and sits at home; it does not yet drive expeditions.	**[Implemented]**. The foundation of everything main-ship — and the chosen home for OSN-2's open-space position.
8 — Expedition stats adapter	`calculate_expedition_stats()` — the single function that turns *ship + captains + modules + loadout + activity* into final stats, enforcing capacity caps (never a plain sum).	**[Implemented]** (read-only; not yet wired into live combat). The "one source of stats" rule.
9A — Visual galaxy map	A real, read-only 2D SVG galaxy map (pan/zoom) showing the world, your home, your ship, and active movements.	**[Implemented]**. The visual substrate OSN-1 later drew the marker onto.
9B — Map-based destination selection	Click a location on the map → pick a loadout → **send an expedition** (reusing the verified send path). The map became the single send surface.	**[Implemented]**.
9C — Expedition UI reframe	Renamed/reframed the UI toward "Ship / Send Expedition / Galaxy Map," removed duplicate send controls. Front end/copy only.	**[Implemented]**.
10A — Transition design doc	Wrote `MAINSHIP_TRANSITION.md`. Docs only.	**[Implemented]** (docs).
10B — Read-only main-ship preview	A read-only "what would my ship bring" view (later simplified to a main-ship-only readout when support craft was deprecated).	**[Implemented]**.
10C — Main-ship send (write path)	`send_main_ship_expedition` + `request_main_ship_return` + a reconciler job + the `mainship_send_enabled` **flag**. **Non-combat destinations only.**	**[Implemented]**, flag-gated off.
10D — Send/recall UI	The galaxy command panel can send and recall the main ship.	**[Implemented]**, flag-gated. *(Interlude: a canonical speed resolver and a NULL-speed recall bugfix landed here.)*
10E — Legacy isolation hardening	Cleanly separated main-ship fleets from the old disposable-fleet path so they can't cross-contaminate. **(NOT the originally-planned combat.)**	**[Implemented]**.
10F — Destroyed / repair safelock	A `destroyed` ship state + a temporary `repair_main_ship()` recovery. **(NOT the originally-planned "deprecate old send.")**	**[Implemented]**. The repair is a *safelock*, not final gameplay (see §9).
10H — Main ship in Command Center	Surfaced the main ship + a Repair button in the Command Center. (No original design-doc entry for "10H.")	**[Implemented]**.
(unnumbered) — Direct location→location move	`move_main_ship_to_location` (migration 0053): a *present* ship can be sent straight from one location to another without recalling home first.	**[Implemented]**, flag-gated.
10G — Multi-ship	Drop the one-ship-per-player limit; multiple ships, groups, larger combat.	**[Future]**, **not built**.

Two things this section must NOT be read as implying:

- **Main-ship combat exists.** It does **not**. The planned combat content of "10E" was never built;

combat is deliberately deferred until after OSN and Repair & Recovery.

- **Repair is final.** It is **not**. The current Repair button is a **temporary instant-Home

safelock** (§9) — instant, free, no consequences — kept only so a destroyed test ship can recover.

---

6. Current main-ship behavior

In practical, player-facing terms, here is everything the main ship can do **today** (with the `mainship_send_enabled` flag on — it is **off** in normal play):

- **Home → named location.** Send the ship from its home base out to a valid named location.
- **Named location A → named location B.** A ship already *present* at a location can be sent

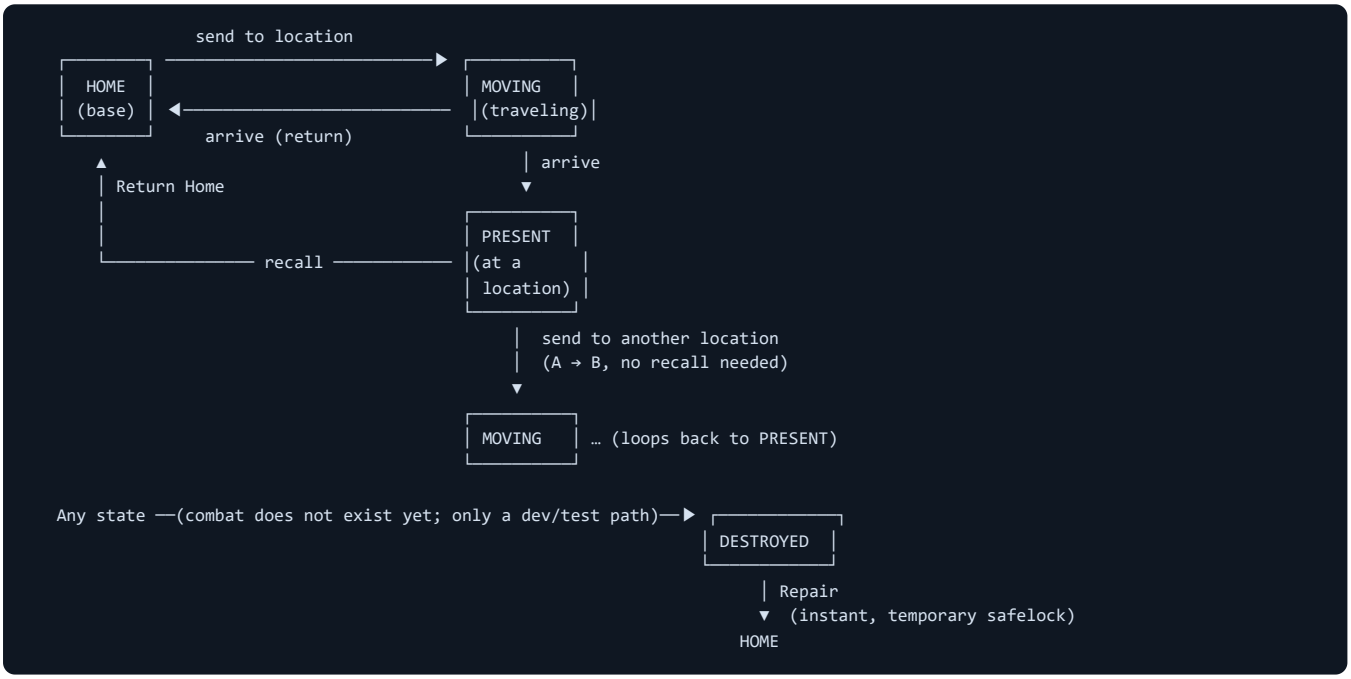
directly to another valid location — **no need to return home first**.

- **Return Home.** Recall the ship back to its home base. (Optional — not a prerequisite for moving.)
- **It cannot receive a new destination while moving, returning, or destroyed.** You can only

redirect it from a stable state (home or present).

- **Destroyed disables normal action.** A destroyed ship can't be sent anywhere.
- **Repair restores it instantly to Home** — the temporary recovery path (see §9).
- **Everything here is non-combat.** No main-ship combat path exists yet.
- **All of this is flag-gated.** With the flag off, none of these surfaces appear to players.

Main-ship state flow (simplified)



> Note the gap this diagram makes obvious: **a ship can only *stop* at a named location.** There is nowhere to "park" in the empty space *between* locations. Closing that gap is exactly what OSN is for.

---

7. Open-Space Navigation (OSN) initiative

OSN is the central next direction. It is a **cross-cutting initiative, deliberately NOT a numbered Phase** — it sits outside both numbering schemes so it collides with neither (§5).

Why named-location travel alone is not enough

Today the ship can only ever be **at** a named location or **traveling between** two of them. But the intended game is about *continuous open space* — exploring unknown coordinates, mining a field that isn't a "location," stopping mid-journey, sitting at a precise point in the void. None of that is expressible when "where is my ship?" can only answer with a menu entry. OSN gives the main ship **one authoritative position model** that supports free movement, stopping in space, and proximity — the foundation that Exploration, Mining, Trading, and (eventually) main-ship combat will all build on.

The OSN stages

For each stage: what it means to the player · its architectural job · what it depends on · and the guardrail (what it must **not** accidentally change).

OSN-1 — Live marker & route visualization [Implemented — closed]

- **Player-facing:** your own main ship shows up as a live marker on the galaxy map, moving along

its route while traveling.

- **Architecture job:** establish **one single, shared, pure position resolver** that every screen

reads from — so position is never computed two different ways. It reads: home → base coords; present → location coords; moving/returning → interpolate along the active movement; destroyed → no marker.

- **Depends on:** the 9A galaxy map and the existing movement data.
- **Must not change:** anything. OSN-1 is **read-only** — no new database tables, functions, flags,

writes, or movement rules. It shows only the **local player's own** ship (no other-player data), though the marker layer is *built to be capable* of more later.

OSN-2 — Durable parked open-space position [Implemented — closed]

- **Player-facing (eventually):** the ship can be *durably parked* at an arbitrary point in empty

space and the game remembers exactly where.

- **Architecture job:** a **durable coordinate** for the "stopped in open space" state, owned by

the **main ship row** (`main_ship_instances`) — see §8 for the full beginner explanation.

- **Depends on:** OSN-1's single resolver (which it *extends*, never duplicates).
- **Must not change:** it does not let the player move freely or stop yet (that's OSN-3/4); it never

fakes a location-presence record for empty space; exactly **one** authoritative source of position at a time.

- **Status detail:** delivered in two steps. **OSN-2a** (migration 0054) added the schema —

nullable `spatial_state` + `space_x/space_y` with finite/pairing CHECKs and **no back-fill** (every existing ship stays legacy `NULL`). **OSN-2b** extended the single resolver to read `in_space` and treat `NULL` as legacy. Verified; flag-gated; no movement writer.

OSN-3 — Arbitrary-coordinate movement [In progress — S1 + S2 + S3 + S4 + S5 done; player UI/Stop are future]

- **Player-facing (eventually):** point at *any* coordinate and fly there — not just named locations.
- **Architecture job:** a parallel, main-ship-only coordinate-movement engine (its own

`main_ship_space_movements` table, **structurally separate** from the frozen legacy `fleet_movements`), travelling from the current point (home / location / parked) to a chosen world coordinate.

- **Depends on:** OSN-2's durable position (a journey can start from, or end as, a parked point).
- **Must not change:** the verified named-location movement functions stay **frozen and canonical** —

OSN-3 runs *alongside* them, never rewriting them.

- **Status detail — S1 [Implemented] (migration 0055), flag `mainship_space_movement_enabled=false` ,

NO writers.** S1 added: the `main_ship_space_movements` route table (`space` / `location` / `base` targets, finite + world-envelope bounds, one-active-per-ship/fleet), the `main_ship_space_command_receipts` idempotency table, the honest moving-fleet pointer `fleets.active_space_movement_id` , the `stationary` status + legacy-safe lifecycle CHECKs, a write-once `fleets.main_ship_id` trigger, and read-model support so the single resolver understands `in_transit` / `at_location` / `home` .

- **Status detail — S2 [Implemented] (migration 0056), flag still `false` , STILL NO writers.** S2 added

the **private, server-only transition boundary** that the future writer will call: four `SECURITY DEFINER` helpers — `mainship_space_lock_context(uuid, boolean)` (locks in the canonical order `main_ship_instances → fleets → main_ship_space_movements → location_presence` , never locks legacy `fleet_movements` , skip-lock at the ship row), `mainship_space_validate_context(uuid)` , `mainship_space_resolve_origin(uuid)` , and `mainship_space_assert_cross_domain_exclusion(uuid)` . All are owned by `postgres` , pinned to `search_path=public` , and granted to `service_role` **only** — `PUBLIC` / `anon` / `authenticated` have no EXECUTE and none is a player-facing RPC. Proven via the real migration chain (`0001..0056`) in a disposable Supabase stack with real concurrent-session lock-order tests, and verified read-only on live (owner/ACL/flags/0-row-count).

- **Status detail — S3 [Implemented] (migration 0057), flag still `false` , the FIRST writer (no public

RPC/UI/processor). **S3 added one private, `service_role`-only, `SECURITY DEFINER` writer `mainship_space_begin_move(p_player uuid, p_main_ship_id uuid, p_target_x double precision, p_target_y double precision, p_request_id uuid)` that composes the S2 boundary (lock → validate → exclusion → resolve-origin) to begin** exactly one **coordinate move in a single transaction: it creates one `moving` `main_ship_space_movements` row from a supported stationary origin (home/legacy_home/in_space materialise a fleet; at_location/legacy_present reuse the present fleet and close its presence), points the fleet's `active_space_movement_id` at it (legacy `active_movement_id` stays NULL), sets the ship `traveling` / `in_transit`, and finalises an idempotency receipt keyed on `(main_ship_id, request_id)`. Target contract is** space-only **(`target_kind='space'`, raw world coords); the client never supplies origin/player/ownership/state/fleet/speed/ETA. Admission is** validate-before-mutate** (every rejection returns `{ok,reason}` with no partial write); the `[-10000,10000]²` envelope is the distance bound and one additive non-flag guard `max_coordinate_travel_seconds=86400` caps travel time. Proven on the real chain (`0001..0057`) — all five origins, full rejection matrix, idempotency, real concurrent-session races, ACL + REST/RPC denial — plus the Build gate and S1/S2 regression, and verified read-only on live (signature/owner/ACL/flags/cap, `main_ship_space_movements` =0, command_receipts=0).

- **Status detail — S4 [Implemented] (migration 0058), flag still `false`, the arrival PROCESSOR (no

public RPC/UI). **S4 added one private, `service_role`-only, `SECURITY DEFINER` processor `process_mainship_space_arrivals()` (no args), driven by a** pg_cron **job `process-mainship-space-arrivals` at the established `30 seconds` cadence (the same convention as `process_fleet_movements`). Each tick it non-lockingly scans due rows (`status='moving'` and `arrive_at<=now()`, oldest first, `LIMIT 100`), claims each ship with `mainship_space_lock_context(id, true)` (skip-locked; S2 canonical lock order; never locks legacy `fleet_movements`), re-validates the `in_transit` context under the lock, and** settles exactly once**: movement `moving → arrived` (`resolved_at`, `terminal_reason='auto_arrival'`, history immutable); fleet `moving → completed` (`location_mode='movement'`, both movement pointers + base fields cleared — the truthful open-space terminal, not a return-to-base); ship `traveling` / `in_transit` → `stationary` / `in_space` at the movement's `target_x` / `target_y` (which the existing map resolver already renders). It** never gates settlement on `mainship_space_movement_enabled` **(so turning the flag off can't strand in-transit ships), never touches `mainship_send_enabled`, creates no receipt/presence, deletes no history, and leaves every contradictory state** untouched** (no settle/fail/repair — hardening deferred to S5). Proven on the real chain (`0001..0058`) — settle-once, idempotency, two-concurrent-settle-once, skip-locked-then-settles, flag-off-still-settles, the seven contradiction no-mutation cases, ACL + REST/RPC denial, cron-present-once — plus the Build gate and S1/S2/S3 regression, and verified read-only on live (processor signature/owner/ACL, one cron job @30s, flags/cap, `main_ship_space_movements` =0, command_receipts=0).

- **Status detail — S5 [Implemented] (migration 0059), flags unchanged, the DESTRUCTION primitive made

coordinate-complete (no public RPC/UI). **S5 re-created** only **the unique trusted destruction writer `dev_set_main_ship_destroyed(p_player uuid)` (still `service_role`-only, `SECURITY DEFINER`, owner `postgres`, no player wrapper, no new cron) so it can destroy a ship in any valid coordinate state. It acquires `mainship_space_lock_context(id, false)` first (canonical order; never locks legacy `fleet_movements`); requires `validate_context` to succeed —** any generic contradiction aborts the whole operation atomically, leaving every row unchanged **(no reconciliation/guessing); for a coherent `in_transit` it cancels the active coordinate movement (`status='cancelled'`, `terminal_reason='ship_destroyed'`, `resolved_at` — history preserved) and clears `active_space_movement_id`; it preserves the existing legacy fleet/presence cleanup; and it sets the ship `destroyed` / `hp=0` / `spatial_state=NULL` /coords NULL. The `NULL` spatial_state (not `'destroyed'`) is deliberate: `repair_main_ship` sets `status='home'` without resetting `spatial_state`, so a repaired ship is a clean `legacy_home` with** no change to `repair_main_ship`**. The S3 command receipt is immutable; no history is deleted. Proven on the real chain (`0001..0059`) — coherent destruction of `in_transit` / `in_space` / `at_location` /legacy, idempotency, real repair-after-destruction, the full contradiction-abort matrix, arrival-vs-destruction concurrency races, ACL + REST/RPC denial — plus the Build gate and S1/S2/S3/S4 regression, and verified read-only on live (primitive signature/ owner/ACL, S4 cron unchanged @30s, flags/cap, counts 0). With S5, the internal coordinate lifecycle is complete and dark:** departure (S3) → arrival settlement (S4) → parked `in_space` → coordinate-complete destruction (S5)**.** Still to come: **a PC-first coordinate command/map surface (a public player wrapper for the writer + target-selection UI, gated by `mainship_space_movement_enabled`), then** OSN-4 Stop** — none of which exist yet.

OSN-4 — Stop mid-travel [Future]

- **Player-facing:** a "Stop" action that halts the ship right where it is, mid-journey.
- **Architecture job:** in **one locked server transaction**, compute the current position from

database time (never the browser clock), persist the stopped coordinate, and cleanly close the old movement — leaving no orphaned or duplicate movement.

- **Depends on:** OSN-2 (somewhere to record the stop) and OSN-3 (free movement to stop within).
- **Must not change:** it must never create a fake location presence or leave a dangling movement.

OSN-5 — Proximity & docking [Future]

- **Player-facing:** being *near* something (a market, a field, a station) is different from being

docked at it.

- **Architecture job:** define "in interaction range" **separately** from "docked / present."

Proximity alone permits nothing; docking is an explicit action.

- **Depends on:** the durable coordinate model (OSN-2) and free movement (OSN-3).
- **Must not change:** proximity must not auto-dock, auto-trade, or auto-fight.

---

8. OSN-2 explained carefully (beginner-friendly)

This is the next real implementation step, so it deserves a slow, plain explanation.

The problem. Right now the game can always answer "where is my ship?" by pointing at *something that already has coordinates*: your home base, a named location, or a movement in progress. But once we let a ship **stop in empty space**, there's a brand-new fact the game has never had to store: a point in the void that isn't a base, isn't a location, and isn't a movement. We need a durable place to write that `(x, y)`.

Why the coordinate belongs on the ship row (`main_ship_instances`). A parked position is a fact *about the ship itself* — "my ship is resting here." The ship's row is **permanent** (it's never deleted; even destruction is just a status), it already exists one-per-ship, and it's already the thing the map marker reads. So the parked coordinate rides naturally on the ship.

Why NOT on fleet rows. A "fleet" in Byeharu is a **transient travel vehicle** — fleet rows reach a finished state and get replaced on the next trip. If we wrote the parked coordinate there, it would be thrown away or orphaned the moment the trip lifecycle moved on. A resting place must not live on a throwaway record. In fact, "the ship is parked" is *defined by the absence of an active fleet* — so the fleet is exactly the wrong owner.

Why NOT a dedicated new table (yet). A separate "spatial state" table would also work, but it would add a new access-control surface and a join to every reader for **no benefit** today — the ship row is already one-per-ship and already read by the resolver. Keep it simple until there's a real reason not to.

`status` vs. `spatial_state` — two different questions. The ship already has a `status` field, but that answers *"what is the ship doing?"* (home, traveling, repairing, destroyed…). **Where** the ship is, is a *separate question*. So OSN-2 introduces a separate selector (call it `spatial_state`) that answers only *"which kind of place is the ship's position read from?"* These two axes are kept independent so they never get confused.

Crucial subtlety — `spatial_state` is a *selector*, not a license to lie. **It does not** mean the ship row gets to override live truth. For *home*, *at a named location*, and *in transit*, the real supporting record (the base, the present fleet + genuine presence, the active movement) is still the authority and must be validated. **Only `in_space`** gives the ship row its *own* directly-stored coordinate — because empty space has no other record to point at. Stale ship-row data must never win over an active movement or a real location.

Only `in_space` stores raw coordinates. The raw `space_x` / `space_y` numbers are written **only** when the ship is genuinely parked in open space. In every other state those fields are empty and the position is *derived* from the existing owner.

One authoritative source at a time. This is the golden rule: at any instant, the ship's position comes from **exactly one** place. No duplicate coordinates, no two systems disagreeing, no fake location-presence invented for empty space.

Destroyed ships keep no coordinate (for now). When destroyed, the ship has no map position and no stored coordinate. Whether a wreck should remember *where* it died is a real question — but it is deliberately deferred to the **Repair & Recovery** initiative (§9), not decided here.

Where the ship's position comes from, by situation

Situation	Where position comes from
Home	base coordinates
At named location	location coordinates
Moving / returning	active movement interpolation
Stopped in open space	main ship `space_x` / `space_y`
Destroyed	no position shown

The planned safe implementation order

1. **OSN-2a — schema only.** Add the columns + validation rules to the ship row. No behavior, no UI.

(Schema goes first so the code's data types never pretend columns exist before the database has them.)

1. **OSN-2b — read-model / resolver only.** Teach the single OSN-1 resolver to understand the new

parked state. Still no way to actually park — just the ability to *read* one if it existed.

1. **OSN-3 — arbitrary-coordinate movement.** Now the ship can fly to a free coordinate.
2. **OSN-4 — Stop action.** Now the ship can halt mid-route and *become* parked.

> **Important:** OSN-2 **does not** yet let the player move freely or stop mid-route. It only builds the durable *place to record* a parked position and teaches the map how to read it. Actually *creating* a parked ship is OSN-3 + OSN-4.

---

9. Repair & Recovery initiative

Why today's Repair is intentionally temporary. When a main ship is destroyed, the current `repair_main_ship()` simply — instantly, for free — sets it back to full health at Home. That is a **safelock**: a guardrail so a destroyed (test) ship can always recover and a player can never get permanently stuck. It is explicitly **not** final gameplay. It has no cost, no time, no location, and no consequences, which would make destruction meaningless if shipped as-is.

What final Repair & Recovery must eventually decide (all **[Future]**, several still undecided):

- **Costs** — materials, currency, or a service fee to repair.
- **Time** — a server-authoritative repair *duration* (computed from database time), not instant.
- **Repair facilities** — home-only repair vs. repairing at stations/colonies (depends on docking).
- **Emergency recovery** — the "never permanently stuck" guarantee must survive in some form, even if

normal repair becomes costly/slow.

- **Destruction position** — does a wreck remember *where* it died? (Ties directly to OSN-2's

free-space coordinate model.)

- **Cargo / activity / movement consequences** — what you lose on destruction; what happens to an

in-progress activity or journey.

- **Migration from the safelock** — how today's `destroyed` status + instant repair move to the new

model **without breaking current players** (keep the old path as compatibility until the new one is proven, then retire it).

Where it sits in the order. Repair & Recovery must come **after** OSN establishes a durable free-space position (OSN-2) and proximity/docking (OSN-5), and **before main-ship combat is released** — because combat *causes* destruction, so real repair must exist before ships can be destroyed in play. Until then, the instant-Home safelock stays, unchanged.

---

10. Future gameplay systems

These are the systems the game is heading toward. A recurring theme: **don't build them prematurely**, because each depends on foundations that must exist first — most of all, OSN's shared position model.

Exploration [Future]

- **You do:** scan and discover unknown space → earn data, shards, blueprints.
- **Depends on:** OSN proximity (OSN-5) to "scan when near an unexplored coordinate"; the inventory

+ pending-reward systems already built.

- **Don't build early:** without OSN, "explore" collapses back into "visit a named location," which

isn't exploration at all.

Mining [Future]

- **You do:** navigate to a resource field and extract ore/crystal/cores.
- **Depends on:** OSN movement (OSN-3) + proximity (OSN-5) to reach and work a *coordinate* that

isn't a named location; inventory for the yield.

- **Don't build early:** a field is a *point in space*, not a menu entry — it needs OSN to exist meaningfully.

Trading [Future]

- **You do:** buy low at one market, carry cargo, sell high at another; mind route danger.
- **Depends on:** OSN routes (the path between markets, with danger along the segment); cargo capacity;

the existing reward/inventory plumbing.

- **Don't build early:** trading is fundamentally about *routes through space* — OSN is the substrate.

> **Note:** Exploration, Mining, and Trading are the three "baseline activities." Their exact order among themselves can be revisited — but **all three depend on OSN**.

Online Presence & Visibility [Future]

- **You do:** start to *see* other players' ships near you — carefully, not globally.
- **Depends on:** OSN (a reliable single-ship coordinate + proximity model) **and** the baseline

activities (so we learn what "visibility" should even mean in real play).

- **Don't build early — and this is a hard rule:** visibility comes **after** Exploration/Mining/

Trading work and **before** any player-to-player interaction. The first version is deliberately small: **nearby ships only**, no global all-player map, no full route sharing, likely **sampled/ delayed** positions, with an explicit *relation* (self / ally / neutral / hostile). **Other players must not be shown globally and in real time by default.**

Player interaction [Future]

- **You do:** trade with, ally with, escort, raid, or fight *other players*.
- **Depends on:** Online Presence & Visibility v1 existing first (so these systems don't each invent

their own incompatible position/visibility logic).

- **Don't build early:** without a shared visibility layer, every interaction system would reinvent

"who can see/reach whom" differently.

Main-ship combat [Future]

- **You do:** fight with your actual main ship (not a disposable unit stack).
- **Depends on:** OSN's coordinate/proximity model **and** Repair & Recovery (because combat destroys

ships, and real destruction needs real recovery). Defeat would land in the already-built 10F destroyed/repair safelock, upgraded by Repair & Recovery.

- **Don't build early:** this is *the* thing most often assumed to exist but deliberately deferred —

it must wait for OSN and Repair & Recovery foundations.

Captains / modules / support craft [Future]

- **You do:** equip captains and modules (and, if revived, support craft) to shape your ship's stats

via the one stat adapter (`calculate_expedition_stats`).

- **Depends on:** the Phase 7/8 main-ship + stats foundation (built); inventory as the crafting bridge.
- **Don't build early:** they only matter once the activities that *use* their stats exist.

Rankings [Future]

- **You do:** compete on seasonal leaderboards across combat/trade/explore/mine metrics.
- **Depends on:** stable underlying systems to measure; reads *finalized* result events only.
- **Don't build early:** ranking unstable systems just measures noise.

Outposts / stations / colonies [Future]

- **You do:** progress from a personal outpost toward stations and colonies.
- **Depends on:** a mature economy + location-investment systems; docking (OSN-5).

- **Don't build early:** needs the economic and spatial groundwork beneath it.

---

11. Current recommended implementation order

A realistic forward order from today. (Exact ordering *among* Exploration / Mining / Trading can be revisited — but all three depend on OSN, so OSN comes first.)

1. **Finish OSN-2** — and only after its final schema/read-model review: OSN-2a (schema-only) then

OSN-2b (read-model/resolver).

1. **OSN-3** — arbitrary-coordinate movement.
2. **OSN-4** — Stop mid-travel.
3. **OSN-5** — proximity / docking.
4. **Baseline Exploration / Mining / Trading** — each consuming the OSN position/proximity model.
5. **Online Presence & Visibility v1** — nearby-only, sampled/delayed, relation-tagged.
6. **Player interaction** — trade, alliances, piracy, PvP (only after visibility exists).
7. **Repair & Recovery** — real costs/time/facilities, replacing the safelock.
8. **Main-ship combat** — built on OSN + Repair & Recovery.
9. **Captains / modules / rankings / outpost progression** — layered in as appropriate.

The throughline: **OSN is the foundation**, the baseline activities consume it, visibility precedes interaction, and combat waits for both spatial and recovery groundwork.

---

12. Glossary

- **Main Ship** — your single, persistent ship; the emotional center of the game. One per player

today. Stored in `main_ship_instances`.

- **Fleet** — a **transient** group/vehicle used to carry out a journey or activity. Fleet records

reach a finished state and are replaced; they are *not* a durable home for position.

- **Fleet Movement** — a record of one journey: origin and target coordinates, depart/arrive times,

and speed. The authoritative source of a ship's position *while traveling*.

- **Location Presence** — the fact that a ship/fleet "is at" a **real named location**. Reserved for

genuine locations only — never faked for empty space.

- **Main Ship Instance** — the database row representing your main ship (`main_ship_instances`):

permanent, one per player, the chosen owner of the future open-space position.

- **OSN (Open-Space Navigation)** — the cross-cutting initiative giving the main ship one

authoritative position model, free movement, stop-in-space, and proximity. Stages OSN-1..OSN-5.

- **Spatial State** — a *future* selector field (OSN-2) answering "which kind of place is the ship's

position read from?" (home / at-location / in-transit / in-space / destroyed). Separate from `status`.

- **In Space** — the one spatial state where the ship is **durably parked at a raw coordinate**

(`space_x` / `space_y`) stored on the ship row. It means *only* "parked at this point" — **not** "idle," "mining," "hiding," etc. Those activity meanings are future consumers.

- **Docking** — an **explicit** transition into "present/interacting at" something. Being *near*

(proximity) is deliberately *not* the same as being docked (OSN-5).

- **Server-authoritative** — the server decides all real outcomes; the client only displays results

and requests actions. The core anti-cheat principle.

- **Feature Flag** — a server-side switch (here, `mainship_send_enabled`) that turns a feature on/off without a code change. Off = the feature is hidden from normal play even though the code exists.
- **Safelock** — a temporary guardrail that guarantees recovery so a player can never get permanently stuck. Today's instant-Home `repair_main_ship()` is a safelock, not final gameplay.
- **Resolver** — the **single, shared, pure function** (`resolveMainShipMarker`, from OSN-1) that computes the ship's map position from one authoritative source. There is deliberately only **one**; OSN-2 *extends* it rather than creating a second.

This is a companion guide. For authoritative detail, defer to `ROADMAP.md`, `MAINSHIP_TRANSITION.md`, `ACTIVITIES.md`, `ARCHITECTURE.md`, `SYSTEM_BOUNDARIES.md`, and the dated `DEV_LOG.md`.